
ARQUITECTURA DE BACKEND — MAQUITRACE

Jhon Jader Riascos Angulo

Charly Jhoan Murillo Hernández

Ingeniería de Sistemas, Universidad del Pacífico

Seminario de Actualización

Ingeniero G. Lucio

30 de septiembre de 2026

Arquitectura de Backend — MaquiTrace

Introducción

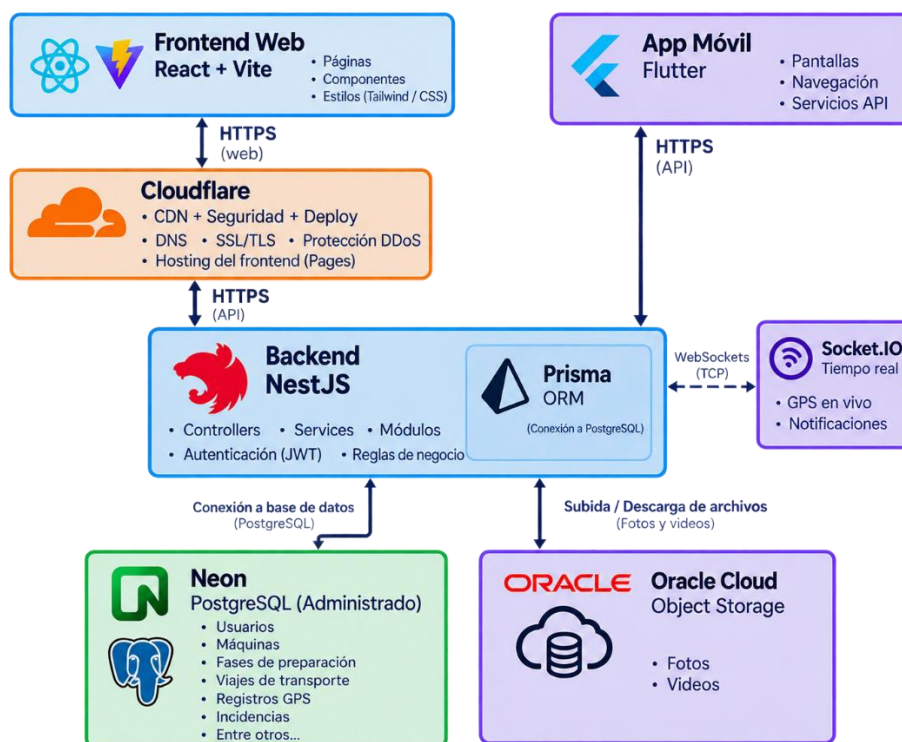
Este documento define la arquitectura de backend acordada para MaquiTrace, un backend propio construido por el equipo, con un framework de patrón MVC, persistencia en PostgreSQL y notificaciones por correo o mediante sockets. Sirve como referencia técnica para el desarrollo durante los seis sprints del proyecto.

Arquitectura general

El siguiente diagrama resume cómo se comunican los componentes del sistema: la aplicación móvil y la plataforma web consumen la API propia; el backend es el único que se comunica con la base de datos y con el almacenamiento de archivos.

Figura 1

Arquitectura general de MaquiTrace



La comunicación en tiempo real de la ubicación GPS se realiza mediante WebSockets mediante Socket.IO, en paralelo a las peticiones HTTP. Ni la aplicación móvil ni la plataforma web acceden directamente a Neon o a Oracle Cloud; todo pasa por el backend.

Stack tecnológico

Componente	Herramienta
Framework backend	Node.js con NestJS (TypeScript)
Base de datos	PostgreSQL
ORM	Prisma
Autenticación	JWT mediante Passport (@nestjs/passport)
Tiempo real	Socket.IO (@nestjs/websockets y @nestjs/platform-socket.io)
Almacenamiento de evidencias	Oracle Object Storage, accedido únicamente desde el backend
Notificaciones por correo	Nodemailer
Despliegue	VPS propia u Oracle Cloud Free Tier

Patrón MVC aplicado

NestJS no utiliza los términos Modelo, Vista y Controlador de forma literal, pero organiza el código bajo el mismo principio de separación:

- **Modelo:** las entidades definidas en Prisma (Machine, User, PreparationPhase, entre otras) y la lógica de negocio contenida en los Services.
- **Controlador:** los Controllers, responsables de recibir las peticiones HTTP y delegarlas al Service correspondiente.
- **Vista:** al tratarse de una API consumida por la app móvil y la plataforma web, la "vista" es la respuesta en formato JSON devuelta por cada endpoint.

Estructura de carpetas

```

backend/
├── src/
│   ├── auth/           # login, JWT, guards por rol
│   ├── users/          # operarios, transportadores, administradores
│   ├── machines/       # maquinaria por categoría y serial
│   ├── preparation-phases/ # ensamblaje, pintura, lavado
│   ├── evidence/       # fotos y videos (Oracle Object Storage)
│   ├── transport/      # recepción, salida, entrega
│   ├── tracking/       # posiciones GPS y gateway de sockets
│   ├── incidents/      # novedades registradas en ruta
│   ├── notifications/  # correo y eventos en tiempo real
│   ├── prisma/         # esquema y cliente de Prisma
│   └── main.ts
├── prisma/
│   └── schema.prisma
└── package.json
  
```

Cada carpeta bajo src/ corresponde a un módulo de NestJS, compuesto por su controlador, su servicio y su módulo, y se relaciona directamente con las tablas definidas para la base de datos: usuarios, roles, maquinarias, fases_alistamiento, evidencias, viajes_transporte, registros_gps e incidencias.

Endpoints por módulo

Autenticación

```

POST /auth/login
POST /auth/logout
  
```

Usuarios

Gestionados desde la plataforma web, únicamente por el rol Administrador.

```

GET    /users
POST   /users          # crear operario o transportador
PATCH /users/:id
  
```

Maquinaria

```
GET /machines          # lista, con filtros por categoría, estado y serial
GET /machines/:id
POST /machines          # registrar una nueva máquina
GET /machines/:id/history # línea de tiempo completa
```

Fases de alistamiento

```
GET /machines/:id/phases
PATCH /machines/:id/phases/:phaseId # actualizar estado y observaciones
```

Evidencias

```
POST /machines/:id/evidence # sube a Oracle Object Storage y guarda la URL
GET /machines/:id/evidence
```

Transporte

```
POST /transport/receive/:machineId
POST /transport/depart/:machineId
POST /transport/deliver/:machineId
```

Seguimiento (GPS)

```
POST /tracking/:tripId/location # la app envía coordenadas periódicas
GET /tracking/active            # la web consulta las máquinas en tránsito
```

Adicionalmente, un gateway de WebSocket (tracking.gateway.ts) emite el evento location:update cada vez que llega una nueva posición, de modo que el mapa de supervisión se actualiza en tiempo real sin que la plataforma web deba consultar la API de forma repetida.

Incidencias

```
POST /transport/:tripId/incidents
```

Integración con Oracle Object Storage

Oracle Object Storage se utiliza exclusivamente como servicio de almacenamiento de archivos, y no como base de datos del sistema. El flujo es el siguiente: la aplicación envía la fotografía o el video al endpoint correspondiente del backend; el backend, mediante un cliente compatible con S3, sube el archivo al bucket configurado en Oracle Cloud y guarda en PostgreSQL únicamente la URL resultante. Ni la aplicación móvil ni la plataforma web se comunican directamente con Oracle; toda solicitud pasa por la API propia del equipo.

Modelo de datos

A continuación se detallan los campos de las ocho tablas identificadas para la base de datos, alineadas con las entidades de Prisma que las representan en el backend.

usuarios

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
name	varchar	Nombre completo
email	varchar (único)	Correo de acceso
password_hash	varchar	Contraseña cifrada
role_id	UUID (FK roles)	Rol asignado

Campo	Tipo	Descripción
phone	varchar	Teléfono de contacto
created_at	timestamp	Fecha de creación

roles

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
name	varchar (único)	operario, transportador o administrador

maquinarias

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
category	varchar	Categoría (excavadora, cargador, etc.)
serial	varchar (único)	Número de serie
model	varchar	Modelo de la máquina
status	enum	pendiente, en_proceso, completada, en_transito, entregada
created_at	timestamp	Fecha de registro

fases_alistamiento

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
machine_id	UUID (FK maquinarias)	Máquina asociada
name	enum	ensamblaje, pintura o lavado
status	enum	pendiente, en_proceso, completada
observations	text	Observaciones registradas por el operario
operator_id	UUID (FK usuarios)	Operario responsable

Campo	Tipo	Descripción
started_at	timestamp	Inicio de la fase
completed_at	timestamp	Cierre de la fase

evidencias

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
machine_id	UUID (FK maquinarias)	Máquina asociada
phase_id	UUID (FK fases_alistamiento, opcional)	Fase asociada, si aplica
type	enum	foto o video
url	varchar	Ruta del archivo en Oracle Object Storage
uploaded_by	UUID (FK usuarios)	Quién subió la evidencia
created_at	timestamp	Fecha de carga

Cabe aclarar que machine_id en evidencias no es redundante frente a phase_id, pese a que una evidencia asociada a una fase permite llegar a la máquina de forma indirecta. phase_id es opcional, ya que existen evidencias que no pertenecen a ninguna fase (fotografías finales de inspección, evidencias del cargue y despacho, y la fotografía de entrega registrada por el transportador); en esos casos machine_id es la única referencia disponible. Para los registros donde sí existe phase_id, la consistencia entre ambos campos se garantiza a nivel de aplicación: el backend toma el machine_id de la fase asociada en lugar de aceptar el enviado por el cliente.

viajes_transporte

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
machine_id	UUID (FK maquinarias)	Máquina transportada
transporter_id	UUID (FK usuarios)	Transportador asignado
vehicle	varchar	Vehículo utilizado
destination	varchar	Destino del viaje

Campo	Tipo	Descripción
status	enum	pendiente, en_transito, entregado
departure_at	timestamp	Fecha y hora de salida
arrival_at	timestamp	Fecha y hora de llegada

registros_gps

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
trip_id	UUID (FK viajes_transporte)	Viaje asociado
latitude	decimal	Latitud reportada
longitude	decimal	Longitud reportada
recorded_at	timestamp	Momento del reporte

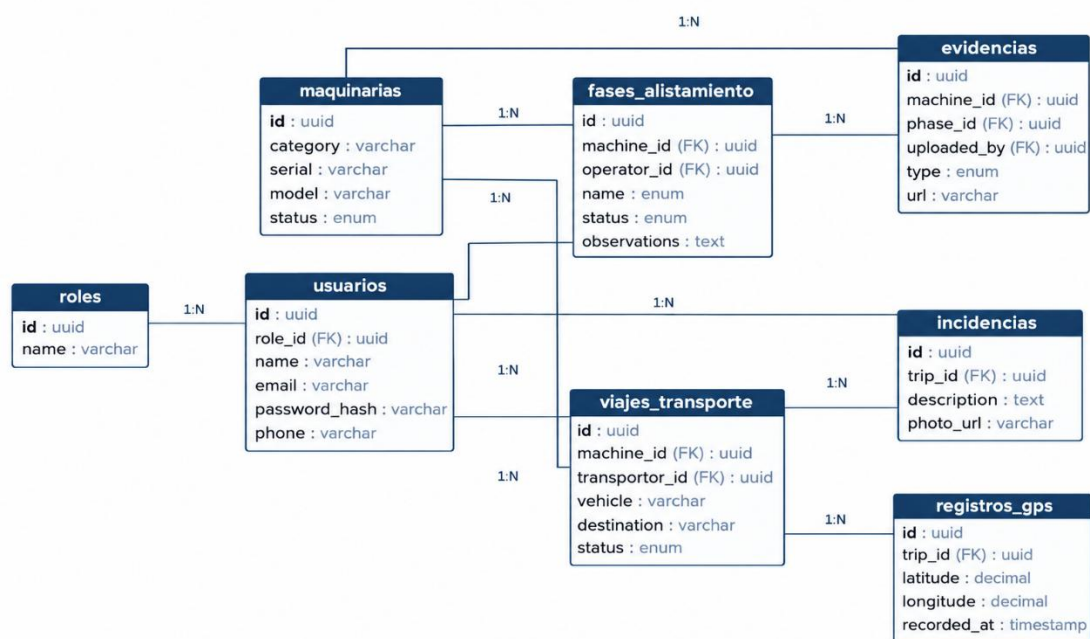
incidencias

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
trip_id	UUID (FK viajes_transporte)	Viaje asociado
description	text	Descripción de la novedad
photo_url	varchar	Fotografía de la incidencia, si se adjunta
reported_at	timestamp	Momento del reporte

Diagrama entidad-relación

Figura 2

Modelo entidad-relación de MaquiTrace



Esquema de Prisma

```

enum MachineStatus {
  pendiente
  en_proceso
  completada
  en_transito
  entregada
}
  
```

```

enum PhaseName {
  ensamblaje
  pintura
  lavado
}
  
```

```

enum PhaseStatus {
    pendiente
    en_proceso
    completada
}

enum EvidenceType {
    foto
    video
}

enum TripStatus {
    pendiente
    en_transito
    entregado
}

model User {
    id                String                @id @default(uuid())
    name              String
    email             String                @unique
    passwordHash      String
    phone             String?
    roleId            String
    role              Role                 @relation(fields: [roleId],
references: [id])
    preparationPhases PreparationPhase[] @relation("PhaseOperator")
    evidenceUploads   Evidence[]          @relation("EvidenceUploader")
    transportTrips     TransportTrip[]     @relation("TripTransporter")
    createdAt          DateTime             @default(now())
}

model Role {
    id    String @id @default(uuid())
    name  String @unique // operario | transportador | administrador
    users User[]
}

model Machine {
    id          String                @id @default(uuid())
    category    String
    serial       String                @unique
    model        String
    status       MachineStatus
    phases       PreparationPhase[]
    evidence      Evidence[]
    trips         TransportTrip[]

```

```

    createdAt DateTime          @default(now())
}

model PreparationPhase {
  id          String          @id @default(uuid())
  machineId   String
  machine     Machine         @relation(fields: [machineId], references:
[id])
  name        PhaseName
  status      PhaseStatus
  observations String?
  operatorId  String
  operator    User            @relation("PhaseOperator", fields:
[operatorId], references: [id])
  evidence    Evidence[]
  startedAt   DateTime?
  completedAt DateTime?
}

model Evidence {
  id          String          @id @default(uuid())
  machineId   String
  machine     Machine         @relation(fields: [machineId], references:
[id])
  phaseId     String?
  phase       PreparationPhase? @relation(fields: [phaseId], references:
[id])
  type        EvidenceType
  url         String
  uploadedBy  String
  uploader    User            @relation("EvidenceUploader", fields:
[uploadedBy], references: [id])
  createdAt   DateTime          @default(now())
}

model TransportTrip {
  id          String          @id @default(uuid())
  machineId   String
  machine     Machine         @relation(fields: [machineId], references:
[id])
  transporterId String
  transporter  User            @relation("TripTransporter", fields:
[transporterId], references: [id])
  vehicle      String
  destination   String
  status        TripStatus
  departureAt   DateTime?
}

```

```
    arrivalAt      DateTime?
    gpsRecords     GpsRecord[]
    incidents      Incident[]
}

model GpsRecord {
  id              String          @id @default(uuid())
  tripId          String
  trip            TransportTrip @relation(fields: [tripId], references: [id])
  latitude        Float
  longitude       Float
  recordedAt      DateTime        @default(now())
}

model Incident {
  id              String          @id @default(uuid())
  tripId          String
  trip            TransportTrip @relation(fields: [tripId], references: [id])
  description      String
  photoUrl        String?
  reportedAt      DateTime        @default(now())
}
```